

```

1  /*
2  * File:   Accelerometre.h
3  * Author: maxcolloud
4  *
5  * Created on October 20, 2021, 10:06 AM
6  */
7
8  #ifndef ACCELEROMETRE_H
9  #define ACCELEROMETRE_H
10
11  #ifdef __cplusplus
12  extern "C" {
13  #endif
14
15  #include <stdint.h>
16  #define addr_GPIO_CTRL          0x33          // Configuration Interrupt
17  #define addr_DEV_STAT           0x05
18  #define addr_INTR_CTRL         0x06
19  #define addr_MODE               0x07
20  #define addr_SR                 0x08
21  #define addr_MOTION_CTRL       0x09
22  #define addr_FIFO_STAT         0x0A
23  #define addr_FIFO_R_P          0x0B
24  #define addr_FIFO_W_P          0x0C
25  #define addr_Xout_Ex_L         0x0D
26  #define addr_Xout_Ex_H         0x0E
27  #define addr_Yout_Ex_L         0x0F
28  #define addr_Yout_Ex_H         0x10
29  #define addr_Zout_Ex_L         0x11
30  #define addr_Zout_Ex_H         0x12
31  #define addr_Status            0x13
32  #define addr_INTR_STAT         0x14
33  #define addr_Chip_ID           0x18
34  #define addr_Reset             0x1C
35  #define addr_Range             0x20
36  #define addr_XOFFL             0x21
37  #define addr_XOFFH             0x22
38  #define addr_XGAIN8            0x22
39  #define addr_XGAIN             0x27
40  #define addr_YOFFL             0x23
41  #define addr_YOFFH             0x24
42  #define addr_YGAIN8            0x24
43  #define addr_YGAIN             0x28
44  #define addr_ZOFFL             0x25
45  #define addr_ZOFFH             0x26
46  #define addr_ZGAIN8            0x26
47  #define addr_ZGAIN             0x29
48  #define addr_FIFO_CTRL         0x2D
49  #define addr_FIFO_CTRL_TR      0x2E
50  #define addr_FIFO_INTR         0x2F
51  #define addr_FIFO_CTRL2_SR2    0x30
52  #define addr_COMM_CTRL         0x31
53  #define addr_TF_LSB            0x40
54  #define addr_TF_MSB            0x41
55  #define addr_TF_SB             0x42
56  #define addr_AM_LSB            0x43
57  #define addr_AM_MSB            0x44
58  #define addr_AM_DB             0x45
59  #define addr_SHK_LSB           0x46
60  #define addr_SHK_MSB           0x47
61  #define addr_PK_P2P_DUR_LSB    0x48
62  #define addr_PK_P2P_DUR_MSB    0x49
63  #define addr_TIMER_CTRL        0x4A
64  #define addr_RD_CNT            0x4B
65
66  // GPIO Control Register (Read / Write)
67  typedef enum
68  {
69      GPIO1_INTN1_IAH = 0x04,          // 1 = acitf haut
70      GPIO1_INTN1_IPP = 0x08,          // 0 = open-drain, 1 = push-pull
71      GPIO2_INT2_IAH = 0x40,
72      GPIO2_INTN2_IPP = 0x80,
73      Mask_GPIO_CTRL = 0x33,

```

```

74 } MC3419_GPIO_CTRL;
75
76 // Device Status Register (Read)
77 typedef enum
78 {
79     STANDBY_STAT      = 0x00,
80     WAKE_STAT        = 0x01,
81     RES_MODE         = 0x04,
82     I2C_WDT          = 0x10,
83     OTP_BUSY         = 0x80,
84 } MC3419_DeviceStat;
85
86 // Interrupt enable register (Write)
87 typedef enum
88 {
89     INTR_Desable      = 0x00,
90     TILT_INT_EN       = 0x01,
91     FLIP_INT_EN       = 0x02,
92     ANYM_INT_EN       = 0x04,
93     SHAKE_INT_EN      = 0x08,
94     TILT_35_INT_EN    = 0x10,
95     AUTO_CLR_EN       = 0x40,
96     ACQ_INT_EN        = 0x80,
97     Mask_INT_CTRL     = 0x20,
98 } MC3419_INTR_CTRL;
99
100 // Mode Register (Write Only)
101 typedef enum
102 {
103     SLEEP             = 0x00,
104     STANDBY           = 0x03,
105     WAKE              = 0x01,
106     // I2C_WDT_NEG      = 0x10,
107     // I2C_WDT_POS      = 0x20,
108     Mask_Mode         = 0xF0,
109 } MC3419_Mode;
110
111 // Sample Rate Register (For SPI <= 4MHz) (Read / Write)
112 typedef enum
113 {
114     ODR25             = 0x10,
115     ODR50             = 0x11,
116     ODR62_5          = 0x12,
117     ODR100            = 0x13,
118     ODR125            = 0x14,
119     ODR250            = 0x15,
120     ODR500            = 0x16,
121     ODR1000           = 0x17,
122 } MC3419_SamplRate;
123
124 // Motion Control Register
125 typedef enum
126 {
127     TF_ENABLE         = 0x01,
128     MOTION_LATCH      = 0x02,
129     ANYM_EN           = 0x04,
130     SHAKE_EN          = 0x08,
131     TILT_35_EN        = 0x10,
132     Z_AXIS_ORT        = 0x20,
133     RAW_PROC_STAT     = 0x40,
134     MOTION_RESET      = 0x80,
135 } MC3419_MotionControl;
136
137
138 // FIFO Status Register (Read Only)
139 typedef enum
140 {
141     EMPTY             = 0x01,
142     FULL              = 0x02,
143     TRESH             = 0x04,
144 } MC3419_FIFO_Stat;
145
146 // FIFO Read Pointer Register (Read Only)

```

```

147 typedef enum
148 {
149     FIFO_RD_PTR      = 0x1F,
150     FIFO_RD_PTR5     = 0x20,
151 } MC3419_FIFO_ReadPointer;
152
153
154 // FIFO Write Pointer Register (Read Only)
155 typedef enum
156 {
157     FIFO_WR_PTR      = 0x1F,
158     FIFO_WR_PTR5     = 0x20,
159 } MC3419_FIFO_WPointer;
160
161 // Status Register (Read)
162 typedef enum
163 {
164     TILT_FLAG        = 0x01,
165     FLIP_FLAG        = 0x02,
166     ANYM_FLAG        = 0x04,
167     SHAKE_FLAG       = 0x08,
168     TILT_35_FLAG     = 0x10,
169     FIFO_FLAG        = 0x20,
170     NEW_DATA         = 0x80,
171 } MC3419_Status;
172
173 // Interrupt Satus Register (Read / Write)
174 typedef enum
175 {
176     TILT_INT          = 0x01,
177     FLIP_INT          = 0x02,
178     ANYM_INT          = 0x04,
179     SHAKE_INT         = 0x08,
180     TILT_35_INT       = 0x10,
181     FIFO_INT          = 0x20,
182     ACQ_INT           = 0x80,
183     Mask_INTR_STATUS = 0x60,
184 } MC3419_INTR_Status;
185
186 // Reset Control (Read / Write)
187 typedef enum
188 {
189     Reset             = 0x40, // 0 = normal operation 1 = force reset
190 } MC3419_Reset_CTRL;
191
192 // Range And Scale Control Register (Write)
193 typedef enum
194 {
195     Range2g           = 0x00,
196     Range4g           = 0x10,
197     Range8g           = 0x20,
198     Range16g          = 0x30,
199     Range12g          = 0x40,
200     Mask_Range        = 0x07,
201     Range             = 0x70,
202     LPF_EN            = 0x08,
203     LPF               = 0x07,
204 } MC3419_Range;
205
206 // X-AXIS Digital Offset Register (Write)
207 typedef enum
208 {
209     MASK_XOFFH_DOR    = 0x80,
210 } MC3419_XOFF;
211
212 // X-AXIS Digital Gain Register (Write)
213 typedef enum
214 {
215     MASK_XOFFH8        = 0x80,
216 } MC3419_XGAIN;
217
218 // Y-AXIS Digital Offset Register (Write)
219 typedef enum

```

```

220 {
221     MASK_YOFFH_DOR    = 0x80,
222 } MC3419_YOFF;
223
224 // Y-AXIS Digital Gain Register (Write)
225 typedef enum
226 {
227     MASK_YOFFH8        = 0x80,
228 } MC3419_YGAIN;
229
230 // Z-AXIS Digital Offset Register (Write)
231 typedef enum
232 {
233     MASK_ZOFFH_DOR     = 0x80,
234 } MC3419_ZOFF;
235
236 // Z-AXIS Digital Gain Register (Write)
237 typedef enum
238 {
239     MASK_ZOFFH8        = 0x80,
240 } MC3419_ZGAIN;
241
242
243 // FIFO Control Register (Read / Write)
244 typedef enum
245 {
246     ALL_FIFO_CTRL_DIS  = 0x00,
247     FIFO_EMPTY_INT_EN  = 0x01,
248     FIFO_FULL_INT_EN   = 0x02,
249     FIFO_TH_INT_EN     = 0x04,
250     COMB_INT_EN        = 0x08,
251     FIFO_RESET         = 0x10,
252     FIFO_EN            = 0x20,
253     FIFO_MODE          = 0x40,
254 } MC3419_FIFO_CTRL;
255
256 // FIFO Threshold Register (Read / Write)
257 typedef enum
258 {
259     Mask_FIFO_TH       = 0xE0,
260 } MC3419_FIFO_THRESH_CTRL;
261
262
263 // FIFO Interrupt Status Register (Read Only)
264 typedef enum
265 {
266     EMPTY_INT          = 0x01,
267     FULL_INT           = 0x02,
268     TRESH_INT          = 0x40,
269 } MC3419_FIFO_INTR;
270
271
272 // FIFO Controle Register2, Sample Rate Register2 (Read / Write)
273 typedef enum
274 {
275     DEC_MODE_RATE      = 0x1F,
276     ENABLE_WRAP_N      = 0x10,
277     SELECT_WRAP_ADDR   = 0x20,
278     FIFO_BURST_MODE    = 0x80,
279 } MC3419_FIFO_CTRL_2;
280
281
282 // Communication Control Register (Read / Wrtie)
283 typedef enum
284 {
285     INT1_INT2_REQ_SWAP = 0x10,
286     SPI_3WIRE_EN       = 0x20,
287     INDIV_INTR_CLR     = 0x40,
288 } MC3419_COMM_CTRL;
289
290
291 // TILP/FLIP Threshold register (Write)
292 typedef enum

```

```

293 {
294     Mask_THRESH_TF_MSB = 0x80,
295 } MC3419_TF_THRESH;
296
297 // Anymotion Threshold Register (Write)
298 typedef enum
299 {
300     Mask_THRESH_AM_MSB = 0x80,
301 } MC3419_AM_THRESH;
302
303 // Shake Duration, Peak-To-Peak Register (Write)
304 typedef enum
305 {
306     Mask_PK_P2P_DUR_MSB = 0x80,
307 } MC3419_PK_TRESH;
308
309 // Timer Control register (Write)
310 typedef enum
311 {
312     TILT_35 = 0x07,
313     TEMP_PERIOD = 0x70,
314     TEMP_PER_INT_EN = 0x80,
315 } MC3419_TIMER_CTRL;
316
317 // Fonctions
318 void MC3419_SetMode (MC3419_Mode Mode);
319 void MC3419_SetRangeCtrl (MC3419_Range Range);
320 void MC3419_SetSampleRate(MC3419_SamplRate SampleRate);
321 void MC3419_wake(void);
322 void MC3419_stop(void);
323 void MC3419_reset(void);
324 void MC3419_start(void);
325 uint8_t MC3419_ID (void);
326 uint8_t ReadRegister8(uint8_t address);
327 void writeRegister8 (uint8_t address, uint8_t data);
328 void delay(uint8_t Xms);
329 void MC3419_Pol_INT(MC3419_GPIO_CTRL GPIO_CTRL);
330 void MC3419_INT_Enable (MC3419_INTR_CTRL INTR_CTRL);
331 void MC3419_MotionCTRL (MC3419_MotionControl Motion);
332 uint8_t MC3419_ReadStatusRegister (void);
333 uint8_t MC3419_ReadIntStatusRegisterAndAck (void);
334 uint8_t MC3419_FiFo_CTRL_REG (MC3419_FIFO_CTRL ValCTRL);
335 void MC3419_FIFO_TRESHOLD (uint8_t Fifo_Tresh);
336 void MC3419_WriteShakeThresholdRegister (uint16_t Value);
337 void MC3419_WriteShakeDurationAndP2PRegister (uint8_t shakeDuration, uint16_t
    shakeP2P);
338
339 #ifdef __cplusplus
340 }
341 #endif
342
343 #endif /* ACCELEROMETRE_H */

```